

Tempo Claro — Guia de Integração do Frontend

Documento único de referência para o cliente (Flutter) consumir a API do backend após a migração para o esquema definitivo. Descreve **todos os endpoints existentes**, o que cada um **espera receber**, o que **devolve**, os códigos de status e as mudanças em relação à versão anterior da API.

Gerado a partir do código-fonte real e validado por smoke test de ponta a ponta (registrar → criar hábito → priming → executar → dashboard → stats).

1. Informações gerais

Item	Valor
Base URL	<code>http://<host>:<porta></code> — porta configurada: 8080
Formato	JSON (<code>Content-Type: application/json</code>), UTF-8
Autenticação	JWT Bearer — header <code>Authorization: Bearer <token></code>
Sessão	Stateless (sem cookies); o token vale 24h
CORS	Liberado para qualquer origem (GET, POST, PUT, DELETE, OPTIONS)

⚠ **Porta local:** no ambiente de desenvolvimento atual a porta **8080 pode estar ocupada pela IDE**. Se o backend não subir (`Port 8080 was already in use`), rode com outra porta (`--server.port=8090`) e aponte o `baseUrl` do app para ela. Em produção será 8080.

Rotas públicas vs protegidas

- **Públicas (sem token):** `POST /api/auth/register`, `POST /api/auth/login`.
- **Todas as demais exigem** `Authorization: Bearer <token>`. Sem token válido → **401**.

⚠ Convenção de nomes de campos (importante!)

Os DTOs **misturam** `snake_case` e `camelCase` — use **exatamente** os nomes desta doc, não deduza. Ex.: no mesmo objeto de hábito convivem `meta_base` (snake) e `avatarUrl` (camel).

2. Formato de erro e status codes

Erros de negócio retornam sempre:

```
{ "success": false, "message": "Texto explicativo do erro" }
```

Status	Quando
200 OK	sucesso (GET/PUT/POST de execução, login)

Status	Quando
201 Created	criação (register, criar hábito, iniciar sessão)
400 Bad Request	corpo inválido (@Valid falhou) ou erro genérico
401 Unauthorized	sem token / token inválido ou credenciais de login incorretas
404 Not Found	recurso inexistente (hábito, usuário, sessão, sub-atividade)
422 Unprocessable Content	regra de negócio violada (limite de 2 hábitos, saldo insuficiente, sem escudo, e-mail em uso, sessão duplicada, tipo inválido)

3. Autenticação

POST /api/auth/register — pública

Request

```
{ "nome": "Maria", "email": "maria@exemplo.com", "password": "senha12345" }
```

Response 201

```
{  "token": "eyJhbGciOiJI...",  "user": {    "name": "Maria", "email": "maria@exemplo.com",    "fusoHorario": "America/Sao_Paulo", "preferenciaIdioma": "pt-BR"  } }
```

Erros: 422 e-mail já em uso · 400 validação (nome/email/senha em branco, e-mail inválido). Defaults no cadastro: fusoHorario='America/Sao_Paulo', preferenciaIdioma='pt-BR'.

POST /api/auth/login — pública

Request { "email": "maria@exemplo.com", "password": "senha12345" } **Response 200** — igual ao register (token + user). Credenciais inválidas → 401.

Guardar o token

Use o token em todas as chamadas seguintes via **Authorization: Bearer <token>**.

4. Perfil

PUT /api/profile — protegida

Todos os campos são **opcionais**; envie só o que muda. Para trocar a senha, envie **senha_atual** e **nova_senha**.

```
{ "nome": "Maria S.", "fuso_horario": "America/Bahia",  
  "senha_atual": "senha12345", "nova_senha": "novaSenha456" }
```

Response 200 (sem corpo relevante). Erros: **422** senha atual ausente/incorreta · **404** usuário.

5. Hábitos

GET `/api/dashboard` — protegida

Lista os hábitos **ativos** do usuário, cada um já com os dados de gamificação calculados. **Response 200** — array de **HabitoResponse** (ver schema abaixo).

POST `/api/habits` — protegida

Cria um hábito (wizard F02). Valida o molde e o limite de **2 hábitos ativos**. **Request** — **HabitoRequest**:

Campo	Tipo	Obrig.	Notas
<code>titulo</code>	string	✓	
<code>categoria_codigo</code>	string	✓	AGUA ESTUDO EXERCICIO
<code>meta_base</code>	int	✓	> 0 (ml para Água; segundos para Estudo/Exercício)
<code>tipo_medida</code>	string	✓	QUANTIDADE (Água) TEMPO (Estudo/Exercício)
<code>gatilho_ancora</code>	string	—	intenção de implementação (F03)
<code>horario_agendado</code>	string HH:mm:ss	—	horário-alvo do dia
<code>meta_maxima</code>	int	—	teto da progressão ($\geq$ <code>meta_base</code>)
<code>dias_para_aumento</code>	int	—	progressão: a cada N dias...
<code>incremento_meta</code>	int	—	...soma esse valor à meta
<code>sub_atividades</code>	array	—	partes da meta (ver abaixo)
<code>dias_semana</code>	int[]	—	0=domingo ... 6=sábado

`sub_atividades[]` (cada item): { `"ordem": 1`, `"alvo_parcial": 1000`, `"horario_inicio": "08:00:00"`, `"horario_fim": "12:00:00"` } (`horario_inicio/horario_fim` opcionais; a soma dos `alvo_parcial` deve formar a meta total).

Exemplo (Água, 2000 ml em 2 partes, todos os dias):

```
{  
  "titulo": "Beber água", "categoria_codigo": "AGUA", "tipo_medida":  
  "QUANTIDADE",  
  "meta_base": 2000, "dias_semana": [0,1,2,3,4,5,6],  
  "sub_atividades": [ { "ordem": 1, "alvo_parcial": 1000 },  
    { "ordem": 2, "alvo_parcial": 1000 } ]  
}
```

```
{ "ordem": 2, "alvo_parcial": 1000 } ]
}
```

Response 201 — **HabitoResponse**. Erros: 422 categoria inválida ou limite de 2 atingido.

PUT /api/habits/{id} — protegida

Atualiza campos editáveis. Se **sub_atividades** ou **dias_semana** forem enviados, **substituem** os existentes (delete + re-insert). Mesmo corpo do create (campos não enviados não mudam). **Response 200** { **"success": true** }.

DELETE /api/habits/{id} — protegida

Soft-delete: arquiva o hábito (preserva histórico). **Response 200** { **"success": true** }.

Schema **HabitoResponse**

```
{
  "id": "uuid",
  "categoriaId": "uuid",
  "categoriaCodigo": "AGUA",
  "categoriaNome": "Água",
  "titulo": "Beber água",
  "gatilhoAncora": null,
  "tipo_medida": "QUANTIDADE",
  "horario_agendado": null,
  "meta_base": 2000,
  "meta_maxima": null,
  "dias_para_aumento": null,
  "incremento_meta": null,
  "ativo": true,
  "arquivadoEm": null,
  "sub_atividades": [ { "id": "uuid", "ordem": 1, "alvo_parcial": 1000,
    "horario_inicio": null, "horario_fim": null } ],
  "dias_semana": [ 0, 1, 2, 3, 4, 5, 6 ],

  "saldo": 100,
  "escudosDisponiveis": 0,
  "ofensiva": 1,
  "nivel": 0,
  "avatarUrl": "/assets/avatares/agua/normal_00.png"
}
```

Campos de gamificação (calculados pelo backend, não editáveis):

- **saldo** — moedas do hábito (SUM do ledger).
- **escudosDisponiveis** — escudos comprados menos usados.
- **ofensiva** — streak atual (dias consecutivos cumpridos).
- **nivel** — $(\text{ofensiva} / 10) * 10$ (0, 10, 20, ...).

- `avatarUrl` — asset do avatar para a ofensiva atual (ver §9).

6. Execução, priming e loja

GET `/api/habits/{id}/priming` — protegida

Texto motivacional pré-tarefa (F06). **Response 200** { "texto": "Hora de se hidratar..." }.

POST `/api/habits/{id}/executions` — protegida

Registra a conclusão **ou** falha de uma execução (F07/F08/F10). **Idempotente** por `execution_token` (gere um UUID novo por execução; reenvio do mesmo token → 422 "Execução duplicada").

Request — **ExecutionRequest**:

Campo	Tipo	Obrig.	Notas
<code>execution_token</code>	uuid	✓	idempotência — um novo por execução
<code>tipo</code>	string	✓	COMPLETE_PADRAO COMPLETE_EXTRA FAIL_TIMEOUT FAIL_BLOQUEIO
<code>valor_realizado</code>	int	✓	quanto foi feito (ml ou segundos)
<code>sub_atividade_id</code>	uuid	—	se a execução é de uma parte específica (regra 3.8)

 **Antifraude (RNF08):** para conclusões, o **servidor recalcula** se é padrão (100) ou extra (150) comparando `valor_realizado` com a meta — o cliente **não decide** a recompensa. Envie `COMPLETE_PADRAO` para concluir; o backend promove a `COMPLETE_EXTRA` se $\geq 120\%$. Para desistência use `FAIL_BLOQUEIO` (consume 1 escudo, protege a ofensiva) ou `FAIL_TIMEOUT`. `FAIL_BLOQUEIO` sem escudo disponível → 422.

Response 200 — **ExecutionResponse**:

```
{
  "moedas_ganhas": 100,
  "moedas_totais": 100,
  "dias_seguidos": 1,
  "novo_nivel": 0,
  "escudos_disponiveis": 0,
  "texto_feedback": "Meta batida! Seu corpo agradece cada gota."
}
```

Use `texto_feedback` na tela de Sucesso/Falha (F13); `moedas_ganhas` para a animação de recompensa.

POST `/api/habits/{id}/shield` — protegida

Compra 1 escudo por **1500 moedas** (F12). **Response 200** { "saldo": 0, "escudos": 1 }. Saldo insuficiente → 422.

7. Sessão / Timer (F09)

Persistem o estado do cronômetro para retomar após minimizar o app. Rotas sob `/api/habits/{habitoId}/sessions`.

POST `/api/habits/{habitoId}/sessions`

Inicia o timer. O servidor define `expira_em = início + 1h` (campo `expira_em` no request é ignorado). Já existe sessão viva → 422. **Request** `{ "sub_atividade_id": "uuid-ou-null" }` **Response 201** — **SessaoResponse**:

```
{ "id":"uuid", "habitoId":"uuid", "subAtividadeId":null,
  "iniciadaEm":"2026-06-25T09:00:00-03:00", "pausadaEm":null,
  "valorParcial":0, "estado":"EM_EXECUCAO", "expiraEm":"2026-06-25T10:00:00-03:00" }
```

PATCH `/api/habits/{habitoId}/sessions/{sessaoId}/pause`

Pausa e salva o progresso parcial. **Request** `{ "valor_parcial": 600 }` → **200 SessaoResponse** (`estado:"PAUSADO"`).

PATCH `/api/habits/{habitoId}/sessions/{sessaoId}/resume`

Retoma. **Se já passou de 1h**, o backend registra automaticamente um `FAIL_TIMEOUT` e a sessão volta com `estado:"TIMEOUT"` (trate como falha por tempo). Caso contrário, `estado:"EM_EXECUCAO"`.

GET `/api/habits/{habitoId}/sessions/active`

Sessão viva do hábito para restaurar o timer. **200 SessaoResponse** ou **404** se não houver.

`estado` ∈ `EM_EXECUCAO` · `PAUSADO` · `FINALIZADA` · `TIMEOUT`.

8. Estatísticas (F11)

GET `/api/stats/weekly` — protegida

Constância dos últimos 7 dias por hábito. **Response 200** — array de `StatsWeeklyResponse`:

```
[
  {
    "habitoId": "uuid",
    "titulo": "Beber água",
    "dias": [
      { "data":"2026-06-19", "status":null, "valorTotalDia":0,
        "metaDoDia":2000 },
      { "data":"2026-06-25", "status":"CONCLUIDO", "valorTotalDia":2000,
        "metaDoDia":2000 }
    ],
  },
]
```

```
"recordValor": 2000
}
]
```

dias traz sempre 7 itens (mais antigo → hoje); **status** ∈ CONCLUIDO · PARCIAL · FALHA · null (dia sem registro). **recordValor** = maior **valor_realizado** já alcançado no hábito.

9. Avatar e estados (F05) — como o front deve exibir

- **Nível/evolução (backend):** **avatarUrl** no **HabitoResponse** já vem resolvido para a ofensiva atual (ex.: `/assets/avatares/agua/normal_00.png`). Sobe de tier a cada 10 dias (`normal_00`, `normal_10`, `normal_20`, `normal_30`).
- **Expressão de urgência (frontend):** a expressão (NORMAL / PREOCUPADO / DESESPERADO / CONCLUIDO) é derivada **no cliente** comparando **horario_agendado** + fuso com o relógio:
 - > 2h para o horário → NORMAL
 - < 2h → PREOCUPADO
 - da hora até 1h de atraso → DESESPERADO
 - meta do dia cumprida → CONCLUIDO
- **Convenção de asset:** `/assets/avatares/{molde}/{estado}_{nivel}.png`, com **molde** ∈ `agua|estudo|exercicio`, **estado** ∈ `normal|preocupado|desesperado|concluido|sucesso|falha`, **nivel** ∈ `00|10|20|30`. Os assets estão **catalogados no banco** (tabela `avatares_catalogo`), então o front pode montar a URL localmente trocando só o **estado**.

⚠ Limitação atual: o **avatarUrl** do dashboard retorna sempre o estado **NORMAL**. Para as demais expressões, o front monta a URL trocando o segmento `{estado}` pela convenção acima.

10. Domínios (enums)

Campo	Valores
<code>categoria_codigo</code>	AGUA · ESTUDO · EXERCICIO
<code>tipo_medida</code>	QUANTIDADE · TEMPO
<code>execução tipo</code>	COMPLETE_PADRAO · COMPLETE_EXTRA · FAIL_TIMEOUT · FAIL_BLOQUEIO
<code>dia status</code>	CONCLUIDO · PARCIAL · FALHA
<code>sessão estado</code>	EM_EXECUCAO · PAUSADO · FINALIZADA · TIMEOUT
<code>dias_semana</code>	inteiros 0(dom) ... 6(sáb)

11. Mudanças vs. versão anterior (o que o front precisa adaptar)

Área	Antes	Agora
Criar hábito	<code>categoria</code> (texto livre)	<code>categoria_codigo</code> (<code>AGUA/ESTUDO/EXERCICIO</code>)
Criar hábito	<code>modalidade</code> , <code>meta_frequencia_diaria</code> , <code>intervalo_minutos</code>	removidos ; agora <code>meta_maxima</code> , <code>dias_para_aumento</code> , <code>incremento_meta</code> , <code>sub_atividades[]</code> , <code>dias_semana[]</code>
Hábito (resposta)	<code>moedas_locais</code> , <code>bloqueios_acumulados</code> , <code>execucoes_hoje</code> , <code>proximo_vencimento</code>	removidos ; agora <code>saldo</code> , <code>escudosDisponiveis</code> , <code>ofensiva</code> , <code>nivel</code> , <code>avatarUrl</code>
Execução	cliente definia o sucesso	servidor decide padrão/extra; novo campo opcional <code>sub_atividade_id</code>
Execução (resposta)	<code>novo_nivel</code>	mantém <code>novo_nivel</code> + novos <code>escudos_disponiveis</code>
Loja	retorno simples	retorna { <code>saldo</code> , <code>escudos</code> }
Novo	—	sessões (<code>/sessions/...</code>) e estatísticas (<code>/stats/weekly</code>)

12. Ainda NÃO disponível no backend (planejar no front)

- **GET /api/categories** — **não existe ainda**. Para o wizard, hoje o front precisa ter os 3 moldes fixos (códigos/nome/cor/unidade) hardcoded ou aguardar o endpoint: **AGUA** (Água, ml, `#2EC4F1`) · **ESTUDO** (Estudo, segundos, `#7C5CFC`) · **EXERCICIO** (Exercício, segundos, `#FF8A3D`).
- **Localização**: priming e feedback retornam só **pt-BR** (o backend ainda não usa o idioma do usuário).
- **F14 Notificações push, F16 Onboarding "Medir Dificuldade"**: sem endpoints ainda.
- **F15 Widget**: consome **GET /api/dashboard** (mesmos dados); cache local é responsabilidade do app.
- **GET de hábito individual** não existe; use **GET /api/dashboard** (lista) e filtre por `id`.

Referência viva — atualizar conforme o backend evoluir. Base: especificação funcional definitiva + código-fonte da API.